

Homework #2

CE 6320: Applied Data Structures and Algorithms (Fall 2026)

Department of Electrical and Computer Engineering
University of Texas at Dallas

Instructor: Prof. Mohamed Ibrahim
Due Date: September 29, 2026

Student Name: _____

Student ID: _____

Instructions:

- This homework has **4 problems**.
- You are required to **solve and submit all 4 problems**.
- Each chosen problem will be graded out of its assigned points; your total will be scaled to an overall maximum of **100 points**.
- Show all steps clearly. Partial credit will be awarded.

September 24, 2026

Problem 1: Huffman Coding

[20 points]

Huffman coding builds a *prefix code* so that more frequent symbols have shorter bit strings. The code is a binary tree: each leaf is a symbol; reading from root to leaf with *left edge* = 0 and *right edge* = 1 yields that symbol's code. You will build the tree by repeatedly merging the two smallest weights into a new parent (whose weight is the sum), until one root remains.

Dataset:

A:5, B:9, C:12, D:13, E:16, F:45

“Alive” Subtree Roots: At any moment, we maintain a small list of nodes that are the *current roots of disjoint subtrees still waiting to be merged*. Initially, every leaf is alive:

$$[A(5) \rightarrow B(9) \rightarrow C(12) \rightarrow D(13) \rightarrow E(16) \rightarrow F(45)]$$

Pick the two smallest (5 and 9), merge them to make a new parent 14. Now the alive list holds the *still-unmerged* roots:

$$[14 \rightarrow C(12) \rightarrow D(13) \rightarrow E(16) \rightarrow F(45)]$$

Repeat: always select the two smallest weights from this list, merge, and put the new parent back into the list. Continue until the list has a single node (the final root).

1. Diagram (5 pts). Draw the final Huffman tree for A..F. Label:

- each leaf with its symbol and frequency (e.g., *E:16*),
- each internal node with the sum of its two children,
- small 0/1 markers on left/right edges.

2. C++ Program (9 pts). Keep a *singly linked list* of the alive subtree roots and, at each step, scan it *once* to find the two smallest weights and merge them.

```
struct Node {
    int  weight;      // frequency or sum
    char sym;         // 'A'..'Z' for leaves; 0 for internal
    Node *left, *right;
    Node *next;       // next in the alive list
};

// Push node u at the front of the alive list.
// Note: Passing a reference allows to change the value of the argument
void pushFront(Node* &head, Node* u);

// Find the two smallest-weight nodes in the alive list (single pass).
// Returns pointers to the minima and their predecessors (to splice in O(1)).
void pickTwoMin(Node *head,
                Node* &min1, Node* &min1Prev,
                Node* &min2, Node* &min2Prev);
```

```
// Build the Huffman tree from the given dataset; return the root.
Node* buildHuffman() {
    Node *alive = nullptr;
    // (1) Create leaves A..F and push them into 'alive' (any order).
    // ...
    // (2) Repeatedly pick two smallest, merge, and push parent back
    // ...
    // (3) Return the last remaining node.
    // ...
}

// Pre-order traversal: print "<sym> : <bits>" at leaves.
// buf holds the current path; len is current length.
// Use recursive implementation
void emitCodesPreorder(Node *u, char buf[], int len);

int main() {
    Node *root = buildHuffman();
    char buf[64];
    emitCodesPreorder(root, buf, 0);
    return 0;
}
```

3. Tracking (4 pts). Fill the merge log (weights only is sufficient):

Step	Picked weights	New weight
1	__, __	__
2	__, __	__
3	__, __	__
4	__, __	__
5	__, __	__

Then list the final codes (order A,B,C,D,E,F):

A : ...
B : ...
C : ...
D : ...
E : ...
F : ...

4. Complexity (2 pts). Answer briefly:

- total build time using one scan per merge step to find two minima,
- time to emit codes with a single pre-order traversal.

Problem 2: Count Endpoints in a Communication Network

[20 points]

A campus communication network can be modeled as a binary tree where *internal nodes* are routers/switches (aggregate traffic) and *leaves* are end devices (endpoints). Counting leaves answers: “How many endpoints are currently attached?”—useful for capacity planning.

Topology (Use this topology to answer all questions below). Routers/Switches: R (Core), A1, A2 (Aggregation), E1, E2, E3 (Edge). Endpoints: d1, d2, d3, d4, d5, d6. Parent→children:

$$\begin{aligned} R &\rightarrow (A1, A2), & A1 &\rightarrow (E1, E2), & A2 &\rightarrow (E3, \text{NULL}), \\ E1 &\rightarrow (d1, d2), & E2 &\rightarrow (d3, d4), & E3 &\rightarrow (d5, d6). \end{aligned}$$

1. Diagram (5 pts). Draw the tree above. Label routers/switches and endpoints as shown. Annotate left/right with small (*L*)/ (*R*) markers.

2. C++ Program (9 pts). Implement the minimal functions below. Choose one traversal you studied (pre-, in-, or post-order) to list endpoints.

```
struct Node {
    const char *name; // e.g., "R", "A1", "d1"
    Node *left;       // first downstream link (may be nullptr)
    Node *right;      // second downstream link (may be nullptr)
};

// Create a new node with given name.
Node* newNode(const char *label);

// Build the topology from the above description; return the root ("R").
Node* buildNetwork();

// Return true iff u is an endpoint (leaf).
bool isLeaf(Node *u);

// Count the number of endpoints in the network, “recursively”.
int countLeaves(Node *u);

// List endpoints using your chosen traversal (pick ONE style and stick to it).
void listEndpoints(Node *u);

int main() {
    Node *root = buildNetwork();
    std::cout << "Endpoints (chosen traversal):\n";
    listEndpoints(root);
    int leaves = countLeaves(root);
    std::cout << "Total endpoints: " << leaves << "\n";
    return 0;
}
```

3. Tracking (4 pts). Provide:

- The list of endpoints produced by your traversal.
 - The final count returned by `countLeaves`.
-

4. Complexity (2 pts). Answer briefly:

- Time to compute `countLeaves` on an n -node tree (one traversal over all nodes).
- Auxiliary space due to recursion in terms of the tree height h .

Problem 3: Closest Shared Directory (Lowest Common Ancestor) [20 points]

In a file system, the *closest shared directory* of two paths is the lowest directory that contains both. This is the *Lowest Common Ancestor (LCA)* problem on a directory tree. We will model the file system as a binary tree of directories (each node has up to two subdirectories) and compute LCA using only parent pointers and pointer moves.

Used directory topology.

```
root → (home, var),    home → (alice, bob),  
alice → (docs, pics),  bob → (games, logs),  
var → (tmp, lib).
```

1. Diagram (5 pts). Draw the directory tree. Label each node with its name and optionally mark left/right with small (L) / (R) markers.

2. C++ Program (9 pts). Complete the minimal skeleton below.

```
struct Node {  
    const char *name;  
    Node *left, *right, *parent;  
};  
  
// Create a new node with a given name.  
Node* newNode(const char *s);  
  
// Helper: set L and R as children of p and set their parent pointers.  
void linkLR(Node *p, Node *L, Node *R);  
  
// Build the given tree; return root.  
Node* buildTree();  
  
// Pre-order search by name; return pointer or nullptr.  
Node* findPreorder(Node *u, const char *target);  
  
// Depth in edges from u up to root (root has depth 0).  
int depth(Node *u);  
  
// LCA via parent pointers only.  
// Steps: raise deeper node until both at same depth; then climb both until equal.  
Node* lca(Node *a, Node *b);  
  
int main() {  
    Node *root = buildTree();  
    const char *Q[][2] = {  
        {"docs","pics"},    // expect: alice  
        {"docs","games"},   // expect: home  
        {"logs","tmp"},     // expect: root  
        {"lib","tmp"},      // expect: var  
    };
```

```

        {"alice","games"} // expect: home
    };
    for (int i = 0; i < 5; ++i) {
        Node *a = findPreorder(root, Q[i][0]);
        Node *b = findPreorder(root, Q[i][1]);
        Node *c = lca(a, b);
        std::printf("LCA(%s,%s) = %s\n", Q[i][0], Q[i][1], c ? c->name : "(null)");
    }
    return 0;
}

```

Implementation tips. **findPreorder**: check current node; recurse left then right. **depth**: walk parent pointers counting edges.

3. Tracking (4 pts). For each query pair, write the LCA (name) and briefly show pointer moves (e.g., “raise logs to bob, then climb together”).

Query	LCA (name)	Pointer moves (brief)
(docs, pics)	-----	-----
(docs, games)	-----	-----
(logs, tmp)	-----	-----
(lib, tmp)	-----	-----
(alice, games)	-----	-----

List directories in *level-order* for the final tree.

4. Complexity (2 pts). Answer briefly:

- Time to compute `lca(a,b)` using parent pointers in terms of height h .
- Time to locate a node by name using a pre-order traversal on an n -node tree.

Problem 4: Spell Checker Dictionary with BST

[20 points]

Build a basic dictionary that supports: (i) *insert(word)*, (ii) *contains(word)*, and (iii) *prefix-Count(prefix)*. Store words in a *binary search tree (BST)* ordered lexicographically (*strcmp*). Words are lowercase ASCII.

Dataset (used in all parts). Insert in this order:

$\langle \text{cat, dog, door, dove, cart, car, care, careful, do, zebra} \rangle$

BST rule: if *w* is strictly less than the current node's word, go left; otherwise go right (ties go right).

Mini visualization.

```
      cat
      \
      dog
     /  \
    do   door
        /  \
      dove (zebra later)
(car, cart, care, careful hang under 'cat' ->right via 'd*' / left via 'c*')
```

Note: Your exact shape depends on insertion order; *in-order* traversal always lists words sorted.

1. Diagram (5 pts). Draw the BST *after all insertions*. Label each node with its word. Optionally mark edges with *(L)*/*(R)*. (Only the final tree is required.)

2. C++ Program (9 pts). Implement the minimal functions below.

```
#include <cstring>    // for strcmp/strncmp

struct Node {
    const char *word; // stored as lowercase ASCII
    Node *left, *right;
};

// Create a new node with a given C-string (assume lifetime >= program).
Node* newNode(const char *w);

// Insert word into BST by strcmp (ties to the right).
// Use recursive implementation.
// Return the (new or existing) node.
Node* insert(Node* &root, const char *w);

// Return true iff the word exists in the BST.
bool contains(Node *root, const char *w);

// Return the number of words that start with 'prefix'.
// Hint: recurse into both sides; count this node if strcmp(...)==0.
int prefixCount(Node *root, const char *prefix);
```

```
// In-order print (ascending)
void inorder(Node *u);

int main() {
    Node *root = nullptr;

    // Insert fixed dataset (one call per word):
    insert(root, "cat");
    insert(root, "dog");
    insert(root, "door");
    insert(root, "dove");
    insert(root, "cart");
    insert(root, "car");
    insert(root, "care");
    insert(root, "careful");
    insert(root, "do");
    insert(root, "zebra");

    // Example checks (optional):
    std::cout << "contains('care') = " << contains(root, "care") << std::endl;
    std::cout << "prefixCount('do') = " << prefixCount(root, "do") << std::endl;

    return 0;
}
```

3. Tracking (4 pts). Answer based on your program:

- Does `contains("care")` return true or false? Give the $(L)/(R)$ path from the root.
- Values of `prefixCount("do")`, `prefixCount("car")`, and `prefixCount("z")`.

4. Complexity (2 pts). Answer briefly:

- Time for `insert` and `contains` in terms of height h (comment on average vs. worst case).
- Time for `prefixCount` on an n -node BST; relate to visiting nodes during traversal.